Computer Organization and Architecture Laboratory

MiniRISC Assignment 4: Branch Control and Complete Baseline Processor

Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

Deadline: 14 October 2026

Objective

Complete the **baseline MiniRISC processor** by adding branch execution, PC-selection logic, and a coherent control unit/FSM that coordinates the datapath developed in the earlier assignments.

By the end of this assignment, the processor should support:

ALU operations + immediate operations + `LD/ST` + branches + `NOP` + `HALT`

The previously planned `MUL`/`MAC` instruction remains part of the final ISA, but the Booth multiplier is **not** integrated in this assignment.

1. Baseline Processor Scope

The processor should now integrate:

- Program Counter;
- instruction BRAM ROM;
- instruction decoder;
- register file;
- immediate extraction/extension logic;
- ALU and ALU-input selection;
- data BRAM RAM;
- load/store path;
- branch-decision logic;
- PC-selection logic;
- write-back multiplexer;
- baseline control unit/FSM;
- `NOP` handling;
- `HALT` handling.

2. Branch Capability

The ISA must provide sufficient branch functionality for loops and conditional execution.

At minimum, support the branch operations already chosen by the group in its ISA plan.

Typical choices are:

- unconditional branch;
- branch if zero;
- branch if positive;
- branch if negative.

Equivalent compare-and-branch instructions are acceptable. The exact mnemonics and encoding remain group-defined.

3. Branch Target Calculation

The processor uses **word-addressed instruction memory**. Therefore:

$$PC_{\text{next}} = PC + 1.$$

A convenient PC-relative branch convention is:

$$PC_{\text{target}} = PC_{\text{base}} + \text{Ext}(branch_offset).$$

Each group must clearly define what PC_{base} means.

A group may define the branch relative to the current PC, or relative to the already incremented sequential PC. Either is acceptable if it is documented and used consistently.

4. Branch Decision Logic

The branch unit must evaluate the condition required by the instruction.

For example:

```
BZ R3, offset
```

should take the branch if

$$R3 = 0.$$

A branch-if-negative operation may test:

$$R3[31] = 1.$$

A branch-if-positive convention must be documented clearly, especially whether zero is included or excluded.

5. PC-Selection Logic

The PC must be able to select between at least:

1. sequential next address;
2. branch target;
3. held value for `HALT`.

Conceptually:

$$PC_{\text{next}} = \begin{cases} PC + 1, & \text{normal execution} \\ BranchTarget, & \text{branch taken} \\ PC, & \texttt{HALT} \end{cases}$$

A multiplexer or equivalent next-PC logic should implement this selection.

6. NOP Behavior

During `NOP`:

- no register should be written;
- no data-memory write should occur;
- PC should normally advance to the next instruction;
- no unintended control signal should remain asserted.

7. HALT Behavior

When `HALT` is executed:

- the processor must enter a halted state;
- the PC must stop advancing;
- no further register write should occur;
- no further data-memory write should occur.

The processor may remain halted until reset.

8. Baseline Control Unit / FSM

The control path should now coordinate complete instruction execution.

A multi-cycle implementation may use states such as:

```
FETCH
DECODE
EXECUTE
MEM_READ
MEM_WRITE
WRITEBACK
BRANCH
HALT_STATE
```

These exact state names are not mandatory.

The important requirement is that synchronous instruction/data BRAM timing is handled correctly.

9. Typical Instruction-Level Control

The following table illustrates the expected type of behavior. Exact signal names depend on the group's design.

Instruction	ALU source	Memory write	Reg write	WB source	PC action
ADD	Reg	0	1	ALU	Next
ADDI	Imm	0	1	ALU	Next
LD	Imm	0	1	Memory	Next
ST	Imm	1	0	–	Next
BZ	–	0	0	–	Next/Branch
MOVE	Reg/Pass	0	1	ALU/Pass	Next
NOP	–	0	0	–	Next
HALT	–	0	0	–	Hold

10. Minimum Program for Behavioral Simulation

Create a short program in instruction BRAM that demonstrates arithmetic, memory, branch, `NOP`, and `HALT`.

A suitable logical sequence is:

```
1. Load or create a loop counter.
2. Perform an arithmetic operation.
3. Store a value into data BRAM.
4. Load the value back.
5. Decrement/update the loop counter.
6. Conditionally branch back while the loop condition is true.
7. Execute a NOP.
8. Execute HALT.
```

The exact machine-code words depend on the group's ISA.

11. Required Demonstrations

At minimum, show correct execution of instructions equivalent to:

$$\boxed{ADD,\quad ADDI,\quad LD,\quad ST,\quad MOVE,\quad BZ \text{ or } BPL/BMI,\quad NOP,\quad HALT}$$

Branch Not Taken

Demonstrate one case in which:

$$PC_{\text{next}} = PC + 1.$$

Branch Taken

Demonstrate one case in which:

$$PC_{\text{next}} = BranchTarget.$$

HALT

Show that after `HALT`:

- the PC remains fixed for multiple clock cycles;
- register-file contents remain unchanged;
- memory write enable remains inactive.

12. Signals to Show in the Waveform

Show at least:

- clock and reset;
- current FSM state;
- PC and fetched instruction;
- decoded instruction/opcode;
- source-register values;
- immediate/offset;
- ALU result;
- branch condition, branch target, and branch-taken signal;
- data-BRAM address, read data, and write enable;
- register write enable and write-back data;
- halt indication.

13. Suggested Processor Hierarchy

```
minirisc32_top
|
|-- pc_unit
|-- instruction_bram_rom
|-- instruction_decoder
```

```
|-- control_unit
|-- register_file_16x32
|-- immediate_extension_logic
|-- alu_input_mux
|-- alu32
|-- branch_unit
|-- pc_select_logic
|-- data_bram_ram
‘-- writeback_mux
```

Students may use a different decomposition if the same architectural behavior is clear.

14. Architecture Check

The updated block diagram should clearly show:

- instruction-fetch path;
- register-file read paths;
- immediate path;
- ALU path;
- data-memory path;
- memory-to-register write-back;
- branch-decision path;
- branch-target path back to the PC;
- control signals from the control unit;
- future connection point for the Booth multiplier.

15. Explicitly Not Required on 14 October

The following remain deferred:

- Booth multiplier integration;
- `MUL`/`MAC` execution hardware;
- multiplier operand/destination latches;
- multiplier `start/busy/done` control;
- stall logic for multiplication;
- software-scheduled multiplier NOP timing;
- dot product;
- matrix multiplication;
- image convolution;
- UART result transmission.

16. Submission and Demonstration

Each group should submit/show:

- complete baseline MiniRISC Verilog source;
- instruction BRAM configuration and `.coe` program;
- data BRAM configuration and initialization file;
- behavioral testbench;
- simulation waveform showing the required cases;
- updated processor architecture diagram;
- instruction/control table;
- a short note documenting the group's PC-relative branch convention.

At the end of 14 October, the **baseline MiniRISC processor should be functionally complete** except for the later Booth multiplier extension and final application-level programs.